

Apellido y Nombre: El Barto DNI: _____

TEORÍA

- 1) Un artículo en redes sociales afirma que usar un ORM podría perjudicar la performance de una aplicación por lo tanto NO recomienda su uso. Ud. está de acuerdo? Tiene algo de veracidad la afirmación? Justifique en todos los casos.

Depende del proyecto, es verdad que usar ORM tiene beneficios de velocidad, es legible e incluso puede llegar a quitar código repetido, además de eso es rápido de aprender (Mucho más que SQL). Pero cuando ya cuando el proyecto tiene un entorno de mucho mayor volumen perderá rendimiento, en esos casos es mejor usar SQL puro para optimizar consultas complejas, porque en esos casos ORM tardaría más.

- 2) Asumiendo que existe una variable Contexto EF llamada "context" y que existe una entidad Cliente correctamente creada con EF, se quiere Modificar la razón social de un cliente existente (cuyo id es recibido por parámetro), utilizando el siguiente código, sin lograr el resultado esperado:

```
public ActionResult ModificarRazonSocialCliente(int id)
{
    //Punto b) □ De manera " Modificacion Parcial "
    Cliente var cliente = new Cliente();

    cliente.Id = id;

    cliente.RazonSocial = "Cliente Modificado";

    context.Cliente.Attach(cliente); //Para que le preste atención
    context.Entity(cliente).Property(x=> x.RazonSocial).IsModified=true;

    context.SaveChanges();

    return RedirectToAction(nameof(Index));
}
```

- a) Describa que está sucediendo, es decir identifique y describa el/los problema/s

Esta creando un cliente en la función, guardando su id y razón social. Luego guardas cambios en el contexto, aunque en ningún momento hubo un cambio en el contexto. Por lo que veo intento hacerlo con el método de "Modificación Parcial", además que falta la etiqueta de [HttpPost], por ende, necesita traer desde el contexto al cliente

RTA: CHAT GPT

En el código se crea un objeto nuevo de tipo `Cliente` en memoria, al que se le asigna únicamente el `Id` y la nueva `RazonSocial`. Luego se intenta adjuntar ese objeto al contexto con el método `Attach()`, lo cual le indica a Entity Framework que ese objeto existe en la base de datos, pero no se está cargando ni consultando realmente el registro desde la base. A continuación, se marca la propiedad `RazonSocial` como modificada para que EF actualice solo ese campo y se llama a `SaveChanges()` para guardar los cambios. Sin embargo, hay varios problemas: primero, en el código original hay un error de tipeo en `Attach`, ya que se escribió `Atach` sin doble "t", lo que causará un error de compilación. Además, no se verifica si el cliente con ese `id` existe en la base de datos, por lo que si el registro no está presente la actualización no tendrá efecto. La técnica de modificación parcial usada puede funcionar, pero es delicada, ya que EF asume que el objeto está en estado "Unchanged" salvo por la propiedad marcada como modificada, lo cual requiere que la clave primaria esté correctamente asignada y que el contexto esté bien configurado. También falta manejo de errores o validación para el caso en que el cliente no exista, lo que puede generar fallos en tiempo de ejecución. Por último, falta la etiqueta `[HttpPost]` en el método, que es una buena práctica para acciones que modifican datos, ya que asegura la correcta semántica HTTP y mejora la seguridad.

- b) Corrija el código anterior para que finalmente cumpla el objetivo de modificación.

```
[HttpPost]
public ActionResult ModificarRazonSocialCliente(int id)
{
    // Buscar el cliente en la base de datos
    var cliente = context.Cliente.Find(id);
    if (cliente == null)
    {
        // Manejar el caso si no existe el cliente, por ejemplo mostrar error o redirigir
    }
}
```

```

        return NotFound();
    }

    // Modificar la razón social
    cliente.RazonSocial = "Cliente Modificado";

    // Guardar los cambios
    context.SaveChanges();

    return RedirectToAction(nameof(Index));
}

```

3) A partir del siguiente código:

```

public string LINQ()
{
    //El origen del dato
    int[] fibo = new int[] { 1, 2, 3, 5, 8, 13, 21, 34, 55 };
    //La creacion de la consulta
    var fiboOJO = fibo.Where(n => n >= 8);

    fibo[0] = 100; //Linea 3

    string salida = "";
    //Ejecucion
    foreach (int num in fiboOJO)
    {
        salida += $"{num} - ";
    }
    return salida;
}

```

a) Identifique las partes o pasos de una consulta en LINQ.

LINQ tiene 3 pasos, buscar el origen del dato, consultar el dato (Creación de la consulta) y ejecutarlo

CHAT GPT

En LINQ, una consulta consta de tres pasos principales. Primero, se define el **origen del dato**, que es la colección o fuente sobre la cual se realizará la consulta, en este caso el arreglo **fibo**. Segundo, se realiza la **creación de la consulta**, que consiste en definir la condición o transformación que se quiere aplicar, como el filtro **Where(n => n >= 8)**. Por último, está la **ejecución de la consulta**, que es cuando se recorren los datos y se obtienen los resultados concretos, como en el ciclo **foreach** que concatena los números filtrados en la cadena **salida**.

b) Indique cual es la salida del método y justifique en relación a la Línea 3.

La salida del método será un string en forma de cadena de los números que había en el array "fibo" pero solo los que sean mayor e igual a 8, y cambiando el primero de su fila con un 100
Algo así " 100 - 8 - 13 - 21 - 34 - 55 "

CHAT GPT

Definición de la consulta: La línea **var fiboOJO = fibo.Where(n => n >= 8);** no ejecuta la consulta en ese momento. Simplemente define las condiciones (**n >= 8**) y la fuente de datos (**fibo**). La variable **fiboOJO** guarda la "receta" de la consulta, no el resultado.

Modificación de la fuente de datos: La Línea 3 (**fibo[0] = 100;**) modifica el array *antes* de que la consulta se ejecute. El array **fibo** ahora es { **100**, 2, 3, 5, 8, 13, 21, 34, 55 }.

Ejecución de la consulta: La consulta se ejecuta recién cuando se itera sobre ella, es decir, en el bucle **foreach**. En ese momento, la condición **n >= 8** se aplica sobre el array **ya modificado**. Por eso, el número **100** (que ahora está en la primera posición) cumple la condición y es incluido en el resultado.

Resultado: El resultado que pone ("100 - 8 - 13 - 21 - 34 - 55")

4) Verdadero o falso EF. Justifique en los casos que sea falso:

- Luego de recuperar un objeto del contexto de EF, el mismo tiene estado "**recovered**", ya que está vinculado al contexto.
Falso, recibe el nombre de "Unchanged"
- Una de las limitaciones de EF Core 5 es que NO existe forma de agregar validaciones DataAnnotations a las entidades.
Falso, se puede agregar validaciones haciendo clases parciales de la entidad misma
- En EF Core 5 es mandatorio/obligatorio usar el enfoque DataBaseFirst, debido a que ningún otro enfoque tiene soporte por Microsoft.

Se pueden usar varios tipos de enfoques, como Code First y Database First, pero no Model First (En Core 6 sí)

- d) LazyLoading es el patrón o mecanismo que permite postergar la inicialización de objetos relacionados hasta el momento que son utilizados y se encuentra activo por defecto en Net 5 / EF Core 5.

Lo primero es correcto, excepto que no está activo por defecto

Falso. El patrón está bien definido, pero no está activo por defecto en EF Core 5; debe activarse explícitamente.

5) Verdadero o falso Web Services y Sesión. Justifique en los casos que sea falso:

- a) ASPNET Web API se basa en convenciones respecto al nombre de sus métodos y NO permite bindear parámetros complejos (clases)
Falso, permite bindear tanto primitivos (int,string,etc), como también complejos (En este caso clases)
- b) Las variables de sesión en ASP.NET Core requieren habilitarse explícitamente ya que no están activas por defecto como en ASP.NET 4.x
Verdadero, en ASP.NET Core la sesión no está activa por defecto y hay que habilitarla
- c) Un cliente que invoca a un servicio ASPNET MVC Web api debe usar serialización XML.
Falso, ASPNET MVC Web tiene por default formato JSON, si quiere serializarlo con XML tiene que cambiarlo
Por defecto usan JSON; XML es opcional y debe configurarse.
- d) ASPNET MVC Web Api NO mantiene el estado entre llamadas cliente y el servidor.
Verdadero. Por ser REST, es stateless y no mantiene estado.